

Quick-Start para Nuevos Desarrolladores — DocumentIA

Esta guía describe cómo poner el proyecto en marcha localmente y ejecutar una primera clasificación.

0. Prerequisites

Antes de empezar, verifica que tienes instalado:

- **Windows 11** (o Windows 10)
- **PowerShell 5.1+** (viene con Windows)
- **Git** → git-scm.com
- **.NET 8 SDK** → dotnet.microsoft.com/download
- **Visual Studio Code** (opcional pero recomendado) → code.visualstudio.com
- **Python 3.9+** (solo si vas a entrenar modelos IA)

Verifica tu instalación:

```
dotnet --version
git --version
python --version
```

Si todos responden con sus versiones, puedes continuar.

1. Clone & Setup

1.1 Clonar el repositorio

```
cd C:\Users\tu-usuario\Documents
git clone https://github.com/tu-org/documento-ia-clasificacion-mvp.git
cd documento-ia-clasificacion-mvp
```

1.2 Crear archivo .env local

```
# Copiar plantilla
Copy-Item .env.example .env
```

Abre `.env` en tu editor favorito y completa los valores obligatorios:

```
# Estos son los mínimos para desarrollo local:
AZURE_SUBSCRIPTION_ID=<tu-suscripcion-uuid>
AZURE_TENANT_ID=<tu-tenant-uuid>

# Para dev local, Azurite (emulador) es suficiente:
AzureWebJobsStorage=UseDevelopmentStorage=true
AzureStorageConnectionString=UseDevelopmentStorage=true

# SQL local (si tienes SQL Server corriendo):
SqlConnection=Server=localhost,1433;Database=DocumentIA;User Id=sa;Password=<PASSWORD>;TrustServerCertificate=True;

# Los endpoints de IA pueden dejarse vacíos por ahora (fallback a mock):
Classification__AzureDocumentIntelligence__Endpoint=
Extraction__AzureContentUnderstanding__Endpoint=
```

Nota: Valores de Azure (suscripción, tenant, etc.) se pueden obtener con `az account show` si tienes Azure CLI.

1.3 Restaurar dependencias

```
# Restaurar paquetes .NET
dotnet restore

# (Opcional) Instalar herramientas Python para tests
pip install -r src/ai-models/requirements.txt
```

Esperado: Sin errores de build. Si hay error de compatibilidad .NET, verifica que tienes .NET 8 instalado.

2. Estructura de Carpetas Clave

```
documento-ia-clasificacion-mvp/
├── src/
│   ├── backend/
│   │   ├── DocumentIA.Functions/           ← Main API (Azure Functions)
│   │   │   ├── Program.cs                 ← Startup config
│   │   │   ├── IngestDocument.cs         ← Endpoint POST /api/IngestDocument
│   │   │   └── DocumentIA.Functions.csproj
│   │   └── frontend/
│   │       ├── DocumentIA.Admin/         ← Admin UI (ASP.NET)
│   │       └── ai-models/
│   │           └── training/             ← Scripts de entrenamiento (Python)
│   └── docs/
│       ├── 03_DISEÑO_TECNICO_DETALLADO.md ← ← TU ESTÁS AQUÍ
│       ├── GLOSSARIO_TERMINOS.md        ← Lee esto después
│       ├── Vocab del proyecto
│       └── especificaciones/
│           └── DATA_MODELS_ER_DIAGRAM.md ← Schema de BD
│   └── scripts/
│       ├── legacy/start-azurite.ps1      ← Inicia Storage emulator
│       └── setup/                        ← Setup helpers
│   └── tests/api-tests/
│       ├── test-requests.http            ← E2E tests PowerShell/HTTP
│       ├── REST Client requests
│       └── test-ingest-notasimple.ps1     ← Full E2E test
```

3. Arrancar Localmente

3.1 Iniciar Azurite (Storage Emulator)

Azurite emula Azure Blob Storage + Table Storage localmente.

```
# Abrir nueva terminal PowerShell en el repo root:
.\scripts\legacy\start-azurite.ps1
```

Esperado: Verás en consola:

```
Azurite Blob service is listening at http://127.0.0.1:20000
Azurite Table service is listening at http://127.0.0.1:20002
Azurite Queue service is listening at http://127.0.0.1:20001
```

Déjalo corriendo. Abre otra terminal para el siguiente paso.

3.2 Iniciar Azure Functions

```
# (En la terminal nueva, en repo root)
cd src/backend/DocumentIA.Functions
func start --csharp
```

Esperado: Tras ~10 segundos verás:

```
Now listening on: http://0.0.0.0:7071
Hit CTRL+C to exit...
```

Tu API está viva en `http://localhost:7071`

3.3 Verificar Health Check

En una **tercera terminal**:

```
# Test simple para confirmar que funciona:
curl http://localhost:7071/api/health

# O con PowerShell nativo:
Invoke-WebRequest -Uri "http://localhost:7071/api/health" -Method GET
```

Esperado: Respuesta HTTP 200 con JSON:

```
{
  "status": "healthy",
  "timestamp": "2026-06-10T10:30:45Z"
}
```

¡Sistema está listo!

4. Tu Primer Clasificador

Vamos a ingestar un documento PDF y dejarlo clasificar automáticamente.

4.1 Usar REST Client (Recomendado en VS Code)

1. Instala extensión REST Client:

- VS Code → Extensions (Ctrl+Shift+X)
- Busca `REST Client` (por Huachao Mao)
- Instala

2. Abre archivo `tests/api-tests/test-requests.http`

3. Ejecuta la primera prueba: "1. Ingestar documento de prueba - Tasación"

- Click en el enlace **"Send Request"** sobre el bloque

4. Espera la respuesta:

- Verás en panel derecho un JSON con `instanceId`
- **Copia ese instanceld** (ej: `@instanceId = 12345-abcde-67890`)

5. Ejecuta segunda prueba: "2. Consultar estado de la orquestación"

- El instanceld se sustituye automáticamente
- Verás estado de la clasificación en tiempo real

4.2 Alternativa: PowerShell (Sin VS Code)

Si prefieres línea de comandos:

```
# Crear archivo temporal con JSON de prueba
$body = @{}
    instrucciones = @{}
        expectedType = "Tasacion"
        skipDuplicateCheck = $false
        classification = @{}
            model = "auto"
            umbral = 0.85
        }
    }
    documento = @{}
        name = "test_tasacion.pdf"
        content = @{}
            base64 = "JVBERi0xLjQK..." # PDF mínimo en base64
        }
    }
    trazabilidad = @{}
        correlationId = "TEST-QUICK-001"
        submittedBy = "dev@local"
        idActivo = "ACT-DEV-001"
    }
} | ConvertTo-Json -Depth 10

# Enviar request
$response = Invoke-WebRequest -Uri "http://localhost:7071/api/IngestDocument" `
    -Method POST `
    -ContentType "application/json" `
    -Body $body

$instanceId = ($response.Content | ConvertFrom-Json).instanceId
Write-Host "Documento ingestado. InstanceId: $instanceId"

# Consultar estado (espera 5-10 segundos y vuelve a ejecutar)
$status = Invoke-WebRequest -Uri "http://localhost:7071/runtime/webhooks/durabletask/instances/$instanceId" `
    -Method GET | ConvertFrom-Json

Write-Host "Estado: $($status.runtimeStatus)"
Write-Host "Resultado: $($status.output | ConvertTo-Json)"
```

4.3 Resultado Esperado

```
{
  "runtimeStatus": "Completed",
  "output": {
    "clasificacionResultado": {
      "clasificacion": "Tasacion",
      "confianza": 0.94,
      "detalles": {
        "familia": "Inmuebles",
        "tipo": "Tasacion"
      }
    },
    "timeline": {
      "actividadesEjecutadas": [
        "DocumentoIngestion",
        "Classification",
        "Extraction",
        "Normalization"
      ],
      "duracionPorActividad": {
        "Classification": "2.34s",
        "Extraction": "5.12s"
      }
    }
  }
}
```

¡Acaba de ejecutarse la orquestación completa!

5. Debugging & Tips de Productividad

Breakpoints en VS Code

1. Abre `src/backend/DocumentIA.Functions/IngestDocument.cs`
2. Haz clic en el margen izquierdo (número de línea) para marcar un breakpoint
3. Cuando la ejecución llegue a esa línea, VS Code se pausará automáticamente
4. Inspecciona variables en el panel "Variables" (izquierda)
5. Usa "F5" para continuar ejecución

Application Insights (Observabilidad)

Si tienes Azure accesible, verás telemetría en:

- Azure Portal → Application Insights → sbainprodcai
- Busca tus requests por `correlationId` (que enviaste en trazabilidad)

Ver Logs Locales

Azurite y Functions guardan logs en consola. Si necesitas persistencia:

```
# Todas las tablas en Azurite
$storageContext = New-AzStorageContext -Local
Get-AzStorageTable -Context $storageContext

# Queries en BD local (si SQL está corriendo):
sqlcmd -S localhost,1433 -U sa -P <PASSWORD> -Q "SELECT TOP 10 * FROM [DocumentIA].[dbo].[Executions]"
```

Puertos en Uso

Si port 7071 ya está en uso:

```
# Ver qué proceso usa el puerto:
netstat -ano | Select-String ":7071"

# Matar el proceso (PID):
Stop-Process -Id <PID> -Force

# O cambiar puerto en func start:
func start --csharp --port 7072
```

Errores Comunes

Error	Solución
dotnet: command not found	Reinstala .NET 8 SDK desde dotnet.microsoft.com
Port 7071 already in use	Stop-Process -Name azurite -Force; Stop-Process -Name func -Force
Azurite failing to start	Verifica que puerto 20000-20002 estén libres
SqlConnection error	Verifica SQL Server está corriendo; o deja vacío para usar InMemory DB
Base64 decoding fails	Usa <code>[Convert]::ToBase64String([System.IO.File]::ReadAllBytes("tu.pdf"))</code>

6. Documentación relacionada

- **Vocabulario del proyecto:** [docs/GLOSSARIO_TERMINOS.md](#) — Orquestación, Actividad, Instancia, etc.
- **Arquitectura:** [docs/03_DISENO_TECNICO_DETALLADO.md](#) — flujo de documentos por Classify → Extract → Normalize.
- **Modelo de datos:** [docs/especificaciones/DATA_MODELS_ER_DIAGRAM.md](#) — tablas de Executions, Classifications, Extractions.
- **Tests E2E:** `.\tests\api-tests\test-ingest-notasimple.ps1` y los scripts en `tests/api-tests/`.
- **Extensiones VS Code recomendadas:** C# (Microsoft), Azure Tools (Microsoft), REST Client (Huachao Mao), Python (Microsoft).

Preguntas Frecuentes

P: ¿Necesito Azure para desarrollo local?

R: No, Azurite emula Storage y algunas APIs funcionan sin Azure. Para funcionalidades IA avanzadas (Document Intelligence, OpenAI) necesitarás Keys de Azure.

P: ¿Cómo accedo a la BD local?

R: Usa SQL Server Management Studio o Azure Data Studio conectando a `localhost,1433` con usuario `sa`.

P: ¿Puedo debuggear en modo pausa?

R: Sí, con VS Code + C# extension. Marca breakpoints y ejecuta `func start` con debugger activado.

P: ¿Dónde pido ayuda?

R: Contacta al equipo en el canal #development en Teams o revisa las issues en GitHub.

Si algo no funcionó, revisa la sección **Debugging & Tips** o contacta al equipo.